



Práctica 4

PROBLEMAS DE CONCURRENCIA EN SQL SERVER

Bases de Datos Distribuidas

*Peña Armas Karina Michelle
2023640675*

*Luviano Moreno Alondra
2023640692*

*De la Rosa Valdes Itztli Fernanda
2024640062*

Introducción

En esta practica se reproducen problemas de concurrencia usando dos ventanas de consulta de SQL Server Management Studio conectadas a la misma base de datos

AdventureWorks2019. Para evitar modificar directamente las tablas originales, en cada escenario se crea una tabla pequena de prueba a partir de datos reales de AdventureWorks.

Nuestra idea es observar que pasa cuando dos conexiones trabajan al mismo tiempo sobre los mismos datos y despues probar un nivel de aislamiento que evite el problema.

1. Uncommitted Dependency (Dirty Read) - Lectura sucia

Descripcion del problema

Una lectura sucia ocurre cuando una transaccion puede leer un dato que fue modificado por otra transaccion, aunque ese cambio todavia no haya sido confirmado con COMMIT.

El problema es que la transaccion que realizo el cambio todavia puede ejecutar un ROLLBACK. Si eso ocurre, el dato vuelve a su valor anterior, pero la otra conexion ya utilizo un valor que realmente nunca quedo guardado de forma definitiva.

Para comprobar este problema se utilizan dos conexiones en SQL Server: Conexion A y Conexion B.

Ejemplo

Para no modificar directamente los datos originales de AdventureWorks, se crea una tabla de prueba tomando un producto real.

```
USE AdventureWorks2019;
GO

DROP TABLE IF EXISTS dbo.Demo_DirtyRead;
GO

SELECT TOP 1
    ProductID,
    Name,
    ListPrice
INTO dbo.Demo_DirtyRead
FROM Production.Product
WHERE ListPrice > 0
ORDER BY ProductID;
GO

SELECT *
FROM dbo.Demo_DirtyRead;
GO
```

Conexion A

La Conexion A inicia una transaccion y modifica el precio del producto aumentando su valor un 20%. El cambio todavia no se confirma.

```

USE AdventureWorks2019;
GO

BEGIN TRANSACTION;

SELECT
    ProductID,
    Name,
    ListPrice AS PrecioAntes
FROM dbo.Demo_DirtyRead;

UPDATE dbo.Demo_DirtyRead
SET ListPrice = ListPrice * 1.20;

SELECT
    ProductID,
    Name,
    ListPrice AS PrecioTemporal
FROM dbo.Demo_DirtyRead;

/* Dejar la transaccion abierta. */

```

Conexion B

Mientras la transaccion de A sigue abierta, la Conexion B utiliza el nivel de aislamiento READ UNCOMMITTED.

```

USE AdventureWorks2019;
GO

SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
GO

SELECT
    ProductID,
    Name,
    ListPrice AS PrecioLeidoPorB
FROM dbo.Demo_DirtyRead;
GO

```

La Conexion B puede observar el nuevo precio aunque la Conexion A todavia no haya ejecutado COMMIT.

Regreso a la Conexion A

Despues se cancela la modificacion con ROLLBACK y el precio vuelve a su valor original.

```

ROLLBACK TRANSACTION;
GO

SELECT
    ProductID,
    Name,
    ListPrice AS PrecioDespuesRollback
FROM dbo.Demo_DirtyRead;
GO

```

Resultado

La Conexion B alcanza a leer el precio aumentado, pero posteriormente la Conexion A realiza un ROLLBACK. Por lo tanto, B trabaja con un dato que nunca llega a quedar guardado definitivamente en la base de datos. Esto demuestra el problema de Dirty Read o lectura sucia.

- Precio original: valor confirmado en la tabla.
- Conexion A modifica el precio dentro de una transaccion.
- Conexion B lee el precio temporal usando READ UNCOMMITTED.
- Conexion A ejecuta ROLLBACK y el precio vuelve al valor original.

Solucion

Para evitar una lectura sucia se puede utilizar el nivel de aislamiento READ COMMITTED.

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  
GO
```

Con READ COMMITTED, si la Conexion A modifica un registro y mantiene abierta la transaccion, la Conexion B no puede leer ese cambio sin confirmar. La consulta de B espera hasta que A realice COMMIT o ROLLBACK.

Conclusion

En este escenario se observó que READ UNCOMMITTED permite consultar información que todavía no ha sido confirmada. Esto puede provocar que una aplicación utilice datos incorrectos o temporales. Al utilizar READ COMMITTED, SQL Server evita las lecturas sucias, ya que una transacción no puede leer los cambios pendientes realizados por otra transacción.

The screenshot displays two SQL Server query windows. The left window, titled 'ConexionB_Lectura...(KARI\karp)', shows a transaction that reads data from the 'Demo_DirtyRead' table. The right window, titled 'ConexionA_Lec... (KARI\karp)', shows a transaction that updates the 'ListPrice' of Product 514 to 1.20. The Results pane at the bottom shows the data for Product 514, including the Name 'LL Mountain Seat Assembly' and the PrecioTemporal '192.0096'.

ProductID	Name	PrecioAntes	
1	514	LL Mountain Seat Assembly	160.008

ProductID	Name	PrecioTemporal	
1	514	LL Mountain Seat Assembly	192.0096

```

/* =====
CONEXION B

READ UNCOMMITTED permite leer cambios
que otra transaccion aun NO ha confirmado.

Aqui provocamos la LECTURA SUCIA.
===== */

SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
GO

SELECT
    ProductID,
    Name,
    ListPrice AS PrecioLeidoPorB
FROM dbo.Demo_DirtyRead;
GO

```

ProductID	Name	PrecioLeidoPorB
514	LL Mountain Seat Assembly	192.0096

```

/* =====

USE AdventureWorks2019;
GO

BEGIN TRANSACTION;

UPDATE dbo.Demo_DirtyRead
SET ListPrice = ListPrice * 1.20;

SELECT *
FROM dbo.Demo_DirtyRead;

/*
Dejar abierta la transaccion.
NO hacer ROLLBACK todavía.
*/

```

ProductID	Name	ListPrice
514	LL Mountain Seat Assembly	230.4115

2. Inconsistent Analysis (Nonrepeatable Read) - Lectura no repetible

Descripcion del problema

Una lectura no repetible ocurre cuando una transaccion consulta un dato y, antes de volver a consultarlo, otra transaccion modifica y confirma ese mismo dato.

Como resultado, la primera transaccion obtiene dos valores diferentes al leer dos veces la misma informacion. En este caso se utilizan nuevamente dos conexiones: Conexion A y Conexion B.

Ejemplo

Se crea otra tabla de prueba utilizando un producto de AdventureWorks.

```

USE AdventureWorks2019;
GO

DROP TABLE IF EXISTS dbo.Demo_NonRepeatableRead;
GO

SELECT TOP 1
    ProductID,
    Name,
    ListPrice,
    ListPrice AS PrecioOriginal
INTO dbo.Demo_NonRepeatableRead
FROM Production.Product
WHERE ListPrice > 0
ORDER BY ProductID;
GO

SELECT *
FROM dbo.Demo_NonRepeatableRead;
GO

```

Conexion A - Primera lectura

La Conexion A utiliza READ COMMITTED, inicia una transaccion y consulta por primera vez el precio. La transaccion se deja abierta.

```
USE AdventureWorks2019;
GO

SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
GO

BEGIN TRANSACTION;

SELECT
    ProductID,
    Name,
    ListPrice AS PrimeraLectura
FROM dbo.Demo_NonRepeatableRead;

/* Dejar la transaccion abierta. */
```

Conexion B - Modificacion

Mientras A mantiene abierta su transaccion, B modifica el precio del mismo producto. Como B no abre una transaccion manual, el cambio queda confirmado al terminar el UPDATE.

```
USE AdventureWorks2019;
GO

UPDATE dbo.Demo_NonRepeatableRead
SET ListPrice = ListPrice + 10;
GO

SELECT
    ProductID,
    Name,
    ListPrice AS PrecioModificadoPorB
FROM dbo.Demo_NonRepeatableRead;
GO
```

Conexion A - Segunda lectura

Despues se regresa a la Conexion A y se vuelve a consultar exactamente el mismo dato dentro de la misma transaccion.

```
SELECT
    ProductID,
    Name,
    ListPrice AS SegundaLectura
FROM dbo.Demo_NonRepeatableRead;
GO

COMMIT TRANSACTION;
GO
```

Resultado y analisis

La primera lectura de A muestra el precio original. Despues B modifica y confirma el precio. Cuando A realiza la segunda lectura obtiene un valor diferente, aunque ambas consultas pertenecian a la misma transaccion. Esto demuestra una lectura no repetible.

- Conexion A realiza una primera lectura.
- Conexion B modifica y confirma el dato.
- Conexion A realiza una segunda lectura y obtiene un valor diferente.

Solucion con REPEATABLE READ

Para evitar este problema se utiliza REPEATABLE READ. Con este nivel de aislamiento, los registros que una transaccion ya leyo permanecen protegidos hasta que dicha transaccion termina.

Primero se puede regresar el valor de prueba al precio original:

```
USE AdventureWorks2019;
GO

UPDATE dbo.Demo_NonRepeatableRead
SET ListPrice = PrecioOriginal;
GO
```

En la Conexion A se inicia nuevamente el escenario, ahora con REPEATABLE READ:

```
USE AdventureWorks2019;
GO

SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
GO

BEGIN TRANSACTION;

SELECT
    ProductID,
    Name,
    ListPrice AS PrimeraLectura
FROM dbo.Demo_NonRepeatableRead;

/* Dejar la transaccion abierta. */
```

Si la Conexion B intenta modificar el mismo producto:

```
USE AdventureWorks2019;
GO

UPDATE dbo.Demo_NonRepeatableRead
SET ListPrice = ListPrice + 10;
GO
```

la operacion queda esperando mientras la transaccion de A siga abierta. Esto ocurre porque REPEATABLE READ mantiene protegido el registro que A ya leyo.

Mientras B esta esperando, en la Conexion A se realiza la segunda lectura y se termina la transaccion:

```

SELECT
    ProductID,
    Name,
    ListPrice AS SegundaLectura
FROM dbo.Demo_NonRepeatableRead;
GO

COMMIT TRANSACTION;
GO

```

Las dos lecturas de A deben conservar el mismo valor. Una vez que A hace COMMIT, la operacion que estaba esperando en B puede continuar.

Conclusion

En este escenario se comprobo que READ COMMITTED evita las lecturas sucias, pero todavia permite que un dato cambie entre dos consultas realizadas dentro de la misma transaccion. Al utilizar REPEATABLE READ, SQL Server mantiene protegidos los registros que ya fueron consultados hasta que la transaccion termina. De esta forma se evita que otra conexion modifique esos datos entre la primera y la segunda lectura.

The screenshot displays two SQL scripts running in parallel sessions, labeled CONEXION A and CONEXION B.

CONEXION A Script:

```

/* =====
CONEXION A

READ COMMITTED evita Dirty Reads,
pero todavia puede ocurrir una lectura no repetible.
===== */

SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
GO

BEGIN TRANSACTION;

/* PRIMERA LECTURA */

SELECT
    ProductID,
    Name,
    ListPrice AS PrimeraLectura
FROM dbo.Demo_NonRepeatableRead;

/*
NO HACER COMMIT TODAVIA.

```

CONEXION B Script:

```

/* =====
CONEXION B

Cambiamos el precio mientras A todavia
tiene abierta su transaccion.
===== */

UPDATE dbo.Demo_NonRepeatableRead
SET ListPrice = ListPrice + 10;
GO

/* Comprobamos el nuevo precio */

SELECT
    ProductID,
    Name,
    ListPrice AS PrecioModificadoPorB
FROM dbo.Demo_NonRepeatableRead;
GO

```

Results:

Below the scripts, the results of the queries are shown in two tables. The left table shows the result of the first query in CONEXION A, and the right table shows the result of the query in CONEXION B.

ProductID	Name	PrimeraLectura
514	LL Mountain Seat Assembly	133.34

ProductID	Name	PrecioModificadoPorB
514	LL Mountain Seat Assembly	143.34

```

/* =====
CONEXION A

SEGUNDA LECTURA DEL MISMO DATO
=====

SELECT
    ProductID,
    Name,
    ListPrice AS SegundaLectura
FROM dbo.Demo_NonRepeatableRead;
GO

COMMIT TRANSACTION;
GO

```

%

ProductID	Name	SegundaLectura
514	LL Mountain Seat Assembly	143.34

```

UPDATE dbo.Demo_NonRepeatableRead
SET ListPrice = PrecioOriginal;
GO

SELECT *
FROM dbo.Demo_NonRepeatableRead;
GO

```

75 %

	ProductID	Name	ListPrice	PrecioOriginal
1	514	LL Mountain Seat Assembly	133.34	133.34

3. Phantom Read - Lectura fantasma

Descripción del problema

Una lectura fantasma ocurre cuando una transacción ejecuta una consulta que devuelve un conjunto de filas y, antes de repetir la misma consulta, otra transacción inserta una nueva fila que cumple con la condición de búsqueda. Al repetir la consulta, aparece una fila nueva que no estaba en la primera lectura; esa fila es el “fantasma”.

Para comprobarlo se utilizan dos conexiones en SQL Server: Conexión A y Conexión B. En este ejemplo se trabaja con una tabla de prueba construida a partir de AdventureWorks2019 para no modificar directamente Production.Product.

Idea del escenario

- Conexión A consulta productos dentro de un rango de precios usando REPEATABLE READ y deja abierta la transacción.
- Conexión B inserta un producto nuevo cuyo precio está dentro de ese mismo rango.
- Conexión A repite la misma consulta y ahora obtiene una fila adicional.
- Después se repite la prueba con SERIALIZABLE para impedir la aparición de filas fantasma.

Script completo - Conexión A

Este script contiene la preparación, la demostración con REPEATABLE READ y la solución con SERIALIZABLE. Se ejecuta por secciones, siguiendo los comentarios para alternar con la Conexión B.

```
/******
PROBLEMA 3 - PHANTOM READ / LECTURA FANTASMA
CONEXION A
******/

USE AdventureWorks2019;
GO

/******
PASO 1 - PREPARACION

Creamos una tabla de prueba con productos reales.
La consulta del ejercicio usara el rango de precio 20 a 50.
******/

IF @@TRANCOUNT > 0
    ROLLBACK;
GO

DROP TABLE IF EXISTS dbo.Demo_PhantomRead;
GO

SELECT TOP 3
    ProductID,
    Name,
    ListPrice
INTO dbo.Demo_PhantomRead
FROM Production.Product
WHERE ListPrice BETWEEN 20 AND 50
ORDER BY ProductID;
GO

/* Creamos un indice para que el rango de precios quede bien definido */
CREATE INDEX IX_Demo_PhantomRead_ListPrice
```

```

ON dbo.Demo_PhantomRead(ListPrice);
GO

/* Limpiamos productos de prueba por si se repite el ejercicio */
DELETE FROM dbo.Demo_PhantomRead
WHERE ProductID IN (99998, 99999);
GO

SELECT *
FROM dbo.Demo_PhantomRead
ORDER BY ProductID;
GO

/*****
PASO 2 - PROVOCAR PHANTOM READ

REPEATABLE READ protege las filas que A ya leyo,
pero NO evita que aparezcan nuevas filas que cumplan la condicion.
*****/

SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
GO

BEGIN TRANSACTION;

/* PRIMERA LECTURA */
SELECT
    ProductID,
    Name,
    ListPrice
FROM dbo.Demo_PhantomRead
WHERE ListPrice BETWEEN 20 AND 50
ORDER BY ProductID;

/* Contamos cuantas filas existen */
SELECT COUNT(*) AS PrimeraCantidad
FROM dbo.Demo_PhantomRead
WHERE ListPrice BETWEEN 20 AND 50;

/*****
DETENERSE AQUI.

NO hacer COMMIT.

IR A CONEXION B y ejecutar el PASO 3.
B insertara un producto nuevo dentro del mismo rango de precio.
*****/

/*****
PASO 4 - REGRESAR A CONEXION A

Ejecutar DESPUES de que B haya insertado el producto.
*****/

/* SEGUNDA LECTURA: aparece una fila nueva */
SELECT
    ProductID,
    Name,
    ListPrice
FROM dbo.Demo_PhantomRead
WHERE ListPrice BETWEEN 20 AND 50
ORDER BY ProductID;

SELECT COUNT(*) AS SegundaCantidad
FROM dbo.Demo_PhantomRead
WHERE ListPrice BETWEEN 20 AND 50;

COMMIT TRANSACTION;
GO

/*****
RESULTADO:

La segunda consulta devuelve una fila adicional que no aparecia
en la primera lectura.

Esa fila nueva es la FILA FANTASMA.
*****/

```

```

*****/

/******
PREPARAR LA SOLUCION

Quitamos el producto insertado en la primera demostracion.
*****/

DELETE FROM dbo.Demo_PhantomRead
WHERE ProductID IN (99998, 99999);
GO

/******
SOLUCION - SERIALIZABLE

SERIALIZABLE protege tambien el rango de la consulta.
Por eso B no podra insertar una nueva fila dentro del rango
mientras A mantenga abierta la transaccion.
*****/

SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
GO

BEGIN TRANSACTION;

/* PRIMERA LECTURA */
SELECT
    ProductID,
    Name,
    ListPrice
FROM dbo.Demo_PhantomRead
WHERE ListPrice BETWEEN 20 AND 50
ORDER BY ProductID;

SELECT COUNT(*) AS PrimeraCantidad
FROM dbo.Demo_PhantomRead
WHERE ListPrice BETWEEN 20 AND 50;

/******
DETENERSE AQUI.

IR A CONEXION B y ejecutar la parte de SOLUCION.

B intentara insertar otro producto dentro del rango,
pero debe quedarse esperando.
*****/

/******
PASO FINAL - REGRESAR A A MIENTRAS B ESTA ESPERANDO
*****/

/* La segunda lectura debe mostrar exactamente las mismas filas */
SELECT
    ProductID,
    Name,
    ListPrice
FROM dbo.Demo_PhantomRead
WHERE ListPrice BETWEEN 20 AND 50
ORDER BY ProductID;

SELECT COUNT(*) AS SegundaCantidad
FROM dbo.Demo_PhantomRead
WHERE ListPrice BETWEEN 20 AND 50;

COMMIT TRANSACTION;
GO

/******
Cuando A hace COMMIT, B ya puede terminar su INSERT.
SERIALIZABLE evito la lectura fantasma durante la transaccion de A.
*****/

```

Script completo - Conexión B

Este script se ejecuta en la segunda ventana. Primero inserta la fila que produce el Phantom Read y después intenta otra inserción durante SERIALIZABLE para comprobar que queda bloqueada hasta que A termine.

```
/******
PROBLEMA 3 - PHANTOM READ / LECTURA FANTASMA
CONEXION B
******/

USE AdventureWorks2019;
GO

/******
PASO 3 - INSERTAR LA FILA FANTASMA

Antes de ejecutar esto:
A debe haber hecho su PRIMERA LECTURA con REPEATABLE READ
y debe mantener abierta su transaccion.

Insertamos una fila nueva cuyo precio esta dentro del rango 20 a 50.
******/

INSERT INTO dbo.Demo_PhantomRead
(
    ProductID,
    Name,
    ListPrice
)
VALUES
(
    99999,
    'Producto Fantasma',
    35.00
);
GO

/* Comprobamos que B ya inserto la nueva fila */
SELECT *
FROM dbo.Demo_PhantomRead
WHERE ListPrice BETWEEN 20 AND 50
ORDER BY ProductID;
GO

/******
REGRESAR A CONEXION A.

A repetira la misma consulta y encontrara una fila nueva.
Esa es la lectura fantasma.
******/

/******
SOLUCION CON SERIALIZABLE

Antes de ejecutar esta parte:
A debe estar usando SERIALIZABLE,
haber realizado su primera lectura
y mantener abierta su transaccion.
******/

INSERT INTO dbo.Demo_PhantomRead
(
    ProductID,
    Name,
    ListPrice
)
VALUES
(
    99998,
    'Producto Bloqueado',
    40.00
);
GO
```

```

/*****
ESTE INSERT DEBE QUEDARSE ESPERANDO.

SERIALIZABLE protege el rango de precios que A consulto.
Mientras A no termine, B no puede agregar una fila nueva
que cumpla con esa condicion.

Mientras B esta esperando:
regresar a A, hacer la segunda lectura y ejecutar COMMIT.

Cuando A haga COMMIT, este INSERT podra terminar.
*****/

/*****
LIMPIEZA FINAL

Ejecutar cuando el INSERT anterior ya haya terminado.
*****/

DELETE FROM dbo.Demo_PhantomRead
WHERE ProductID IN (99998, 99999);
GO

SELECT *
FROM dbo.Demo_PhantomRead
ORDER BY ProductID;
GO

```

Resultado

En la primera prueba, la Conexión A utiliza REPEATABLE READ y consulta los productos cuyo precio está entre 20 y 50. Mientras la transacción permanece abierta, la Conexión B inserta un nuevo producto con un precio de 35.00. Cuando A repite exactamente la misma consulta, aparece una fila adicional. La fila no existía en la primera lectura, por lo que se considera una fila fantasma.

REPEATABLE READ evita que las filas que ya fueron leídas cambien mientras la transacción sigue activa, pero no protege completamente el rango contra nuevas inserciones. Por eso puede ocurrir un Phantom Read.

Solución con SERIALIZABLE

Al repetir el escenario con SERIALIZABLE, SQL Server protege el rango consultado. Si la Conexión B intenta insertar un producto nuevo cuyo precio está dentro de ese rango, la operación queda esperando hasta que la Conexión A finaliza su transacción. A puede repetir la consulta y obtiene el mismo conjunto de filas.

Conclusión

La lectura fantasma ocurre cuando una transacción repite una consulta y encuentra filas nuevas insertadas por otra transacción. REPEATABLE READ mantiene estables las filas existentes, pero todavía permite la aparición de nuevas filas que cumplan con la condición. SERIALIZABLE evita este problema al proteger también el rango de búsqueda durante toda la transacción.

```

GO

BEGIN TRANSACTION;

/* PRIMERA LECTURA */

SELECT
    ProductID,
    Name,
    ListPrice
FROM dbo.Demo_PhantomRead
WHERE ListPrice BETWEEN 20 AND 50
ORDER BY ProductID;

/* Primera cantidad */

```

	ProductID	Name	ListPrice
1	707	Sport-100 Helmet, Red	34.99
2	708	Sport-100 Helmet, Black	34.99
3	711	Sport-100 Helmet, Blue	34.99
4	712	Producto Fantasma	35.00

```

SELECT COUNT(*) AS PrimeraCantidad
FROM dbo.Demo_PhantomRead
WHERE ListPrice BETWEEN 20 AND 50;

/*****
DETENERSE AQUI
*****/

```

	PrimeraCantidad
1	4

```

DELETE FROM dbo.Demo_PhantomRead
WHERE Name IN
(
    'Producto Fantasma',
    'Producto Bloqueado'
);
GO

/* Comprobar datos finales */

SELECT
    ProductID,
    Name,
    ListPrice
FROM dbo.Demo_PhantomRead
ORDER BY ProductID;
GO

```

	ProductID	Name	ListPrice
1	707	Sport-100 Helmet, Red	34.99
2	708	Sport-100 Helmet, Black	34.99
3	711	Sport-100 Helmet, Blue	34.99